

run_from_raw_standalone 알고리즘 정리

AEC 저-SMI Standalone 4단계 파이프라인(`run_from_raw_standalone.py`) 알고리즘 정리

대상 파일: `code/past/run_from_raw_standalone.py` (2,109 lines)

이 파일은 `main_aec_full_derivation_pipeline_simplified.py` (해석 가능한 4-region 규칙의 "도출 기록")와는 성격이 다른, **원본 xlsx부터 최종 표/그림까지 한 프로세스로 끝까지 재현하는 "standalone" 스크립트**다. 과거에는

`work/aec_lock_smoothed_deesc_gate.py` → `work/aec_region_cnn_pattern_gate.py` → `work/aec_direct_vote_auc_boost.py` → (별도 phenotype 분석 스크립트) 순서로 4개의 독립 파일이었던 것을, "모듈 의존성 없이 이 파일 하나만 있으면 끝까지 재현 가능"하도록 전부 인라인 병합한 것이다 (주석 3-5행: "Standalone 4-stage ... All helper modules are inlined"). 병합 흔적으로 각 단계별 전역/헬퍼 함수 이름에 `COND_`, `LOCK_`, `PATTERN_` (암묵적), `DVOTE_`, `BOOST_`, `FINAL_`, `CHECK_` 같은 접두어가 붙어 있어 이름 충돌 없이 한 파일 안에 공존한다.

주의: 파일 상단 주석(3행)의 실행 예시 `python code/run_from_raw_standalone.py` 는 병합 이전 위치를 가리키는 낡은 문구이며, 실제 파일은 `code/past/run_from_raw_standalone.py` 에 있다. 또한 `LOCK_SCRIPT` / `PATTERN_GATE_SCRIPT` / `DIRECT_VOTE_AUC_SCRIPT` (78-80행)가 참조하는 `work/aec_*.py` 3개 파일은 이 저장소에 더 이상 존재하지 않는다 — 이 파일 자체가 그 내용을 전부 인라인했기 때문에, `run_if_needed()` 의 "fallback으로 원본 스크립트를 재실행" 경로는 실질적으로 `main()` 을 처음부터 끝까지(Stage 1~3) 돌리는 것으로만 충족된다.

1. 큰 그림: 4단계 파이프라인

```
python code/past/run_from_raw_standalone.py
```

`main()` 은 항상 다음 4단계를 **순서대로, 매 단계 시작 전 전역 RNG를 재시딩**하며 실행한다 (`stage_action()` 이 각 단계 함수를 `reset_shared_random_state()` 로 감싸 실행 순서와 무관하게 동일한 fold 분할이 나오도록 보장 — `main_aec_full_derivation_pipeline_simplified.py` 의 "매번 새로 재시딩" 설계와 동일한 이유).

단계	함수	산출 디렉터리	하는 일
1	LOCK_main()	outputs/run_from_raw_standalone/aec_lock_smoothed_deesc_gate/	방대한 후보 feature 은행에서 internal 전용 탐색으로 "k-of-m 완화(de-escalation) 규칙"을 잠금(lock)
2	PATTERN_main()	outputs/run_from_raw_standalone/aec_region_cnn_pattern_gate/	Stage 1이 고른 feature를 근사하도록 4-region CNN을 학습해 "확률" 산출 → 확률을 패턴 게이트로 재탐색
3	BOOST_main()	outputs/run_from_raw_standalone/aec_direct_vote_auc_boost/	Stage 2의 CNN 확률(+ 임상 feature)을 입력으로 다양한 ML 모델을 crossfit해 AUC가 오르는지 탐색
4	FINAL_main()	outputs/run_from_raw_standalone/aec_final_global_quintile_phenotype/	Stage 3에서 고른 점수 (vote_only_logit_l1)로 "임상 고위험군 내부에서 AEC가 표현형을 분리하는가"를 분위수 기반으로 검증 (진단 성능이 아니라 표현형 enrichment 주장)

각 단계는 독립적으로도 실행 가능하지만(예: PATTERN_main() 은 PROB_CACHE npz가 있으면 재학습을 건너뛰), main() 은 항상 1→4를 전부 재실행한다. 마지막에 FINAL_main() 내부에서 CHECK_main() 을 호출해 outputs/run_from_raw_standalone/MD/ 에 재현성 점검 카드 PNG 1장을 추가로 만든다.

1.1 라벨 정의 (다른 파이프라인과 동일)

```
SMI = TAMA / (Height[m])^2
Low-SMI(y=1): 남성 SMI < 45.4, 여성 SMI < 34.4
```

LOCK_load_dataset() (350행)에서 계산.

1.2 네 단계를 관통하는 공통 설계 원칙

- 1. 모든 임계값/컷오프/feature 선택은 internal(gangnam) 코호트에서만 결정되고, external(sinchon)은 평가에만 쓰인다(12절 참고).
- 2. 재현성: 스레드 풀을 1로 고정(OMP_NUM_THREADS 등, 22-24행)해 BLAS/OpenMP 축소 연산의 합산 순서를 고정하고, PyTorch 는 torch.use_deterministic_algorithms(True) + cuDNN 비-벤치마크 모드로 고정한다 — Stage 2의 CNN 학습 점수가 재실행마다 정확히 같은 값이 나오게 하기 위함.
- 3. 단계 간 인터페이스는 CSV/NPZ 파일이다: Stage 1의 locked_gate_features.csv (선택된 feature 이름) → Stage 2가 그 feature들을 다시 계산해 "정답 투표"로 사용 → Stage 2의 direct_vote_probabilities.npz (CNN 확률) → Stage 3이 그 확률로 ML 모델 학습 → Stage 3의 direct_vote_auc_boost_scores.csv (vote_only_logit_l1 컬럼) → Stage 4가 최종 phenotype 분석에 사용.

2. 경로/상수 (Section 0 근처, line 51-83, 321-337, 944-945, 959-973, 1396-1419, 74-75)

변수	값	의미
PROJECT_ROOT	Path(__file__).resolve().parents[2]	code/past/ 에서 2단계 위 = 저장소 루트
OUTPUT_ROOT	outputs/run_from_raw_standalone	이 스크립트 전용 출력 루트 (다른 파이프라인과 섞이지 않음)
COND_SEED	20260629	전역 RNG (임상 fold 분할)의 시드. 다른 파이프라인들과 동일한 값을 재사용.
LOCK_SEED	20260701	Stage 1의 locked_score_auc_table() 내부 combo 로지스틱 회귀 fold의 시드(전역 RNG 와 별개).
BOOST_SEED	20260701	Stage 3의 StratifiedKFold, model_factory, bootstrap 시드 베이스.
DVOTE_SEEDS	[20260701, 20260711]	Stage 2 CNN 학습을 2개의 서로 다른 시드로 반복해 평균 — 단일 학습 런의 우연성을 줄이기 위한 앙상블.
SIGMA	1.0	AEC-128 곡선 가우시안 스무딩 표준편차 (다른 파이프라인과 동일).
OPS	[S80, S82.5, S85, S87.5, S90] (5개)	임상 목표 민감도 5종 — full_derivation 파이프라인의 3종(S80/85/90)보다 촘촘함(2.5%p 간격 추가).
WIDTHS	[0.35, 0.50, 0.70] / LAMBDA_S	[0.25, 0.40, 0.55, 0.70] — Stage 1 branch_gate_score 류 게이트의 폭/가중치 탐색 격자, 다른 파이프라인과 동일 개념.
TOP_FEATURES_FOR_COMBO	18	Stage 1에서 단일 feature 스크리닝 후 combo 탐색에 넘길 후보 수(다양성 필터 적용, 4.4절).
MAX_COMBO_M	4	Stage 1 combo 탐색에서 동시에 묶는 feature 최대 개수(k-of-m 의 m).
MAX_FEATURES_SCREEN	600	수천 개의 후보 feature 중 사전 스크리닝 (prescreen_feature_indices)으로 추릴 상한.
FORMAL_NI_MARGIN / FORMAL_NI_OP	0.05 / "S90"	clinical_vs_aec_assisted_table.png 가 판정에 쓰는 것과 동일한 공식 비열등성 마진 — Clopper-Pearson 단측 95% 상한 이 이 값 이하여야 규칙이 살아남음(Stage 1 combo_search 에서 직접 강제).
REGIONS	R1 76-92 , R2 88-106 , R3 96-118 , R4 90-110 (1-indexed)	Stage 2 CNN의 4개 branch가 보는 넓은 윈도우. Stage 1이 고 큰 정밀 feature(예: slope_around_082_085) 주변을 더 넓게 잡아 CNN이 정확한 경계가 아니라 "그 근방의 모양"을 스스로 배우게 함.
BRANCH_WIDTH / BRANCH_LAMBDA	[0.70,0.50,0.70,0.70] / [0.70,0.70,0.55,0.55]	Stage 2가 "정답 투표"를 만들 때 4개 locked feature 각각에 적용하는 고정 폭/λ (Stage 1에서 탐색된 값이 아니라 CNN 학습 타깃 생성용으로 별도 고정).
DVOTE_CONFIGS	direct_vote_balanced , direct_vote_guarded	Stage 2 CNN 학습 하이퍼파라미터 2세트 (dropout/lr/weight_decay/consensus_weight/non_cpos_weight) — 더 "공격적"(balanced) vs 더 "보수적"(guarded, 높은 dropout:weight_decay:consensus_weight) 버전을 둘 다 학습해 나중에 더 나은 쪽을 채택.
CANDIDATES	13개 (feature_set, model_key) 조합	Stage 3에서 crossfit할 모델 후보 전체 목록(6.1절).

변수	값	의미
BOOT_N	2000	Stage 3 AUC 델타의 페어드 부트스트랩 반복 수.
PRIMARY_Q / SENSITIVITY_Q	0.20 / 0.25	Stage 4 분위수 enrichment의 주 분석(상하위 20%)과 민감도 점검(25%).
AEC_SCORE_COLUMN	"vote_only_logit_l1"	Stage 4가 사용할 Stage 3 산출 점수의 컬럼명 — 임상 feature를 섞지 않은 "vote-only" 점수를 최종 채택(임상 모델과 완전히 독립적인 AEC 신호임을 보장하기 위함으로 보임).

3. 데이터 로딩 & feature 은행 (line 85-319, 345-401)

- `aec_columns()` / `matrix_from_sheet()`: 다른 파이프라인과 동일한 결측 대체 로직(컬럼 중앙값 → 전역 중앙값).
- `LOCK_load_dataset()`: `raw` → 가우시안 스무딩(`smooth_raw`) → `LOCK_row_norm()` (환자별 평균 정규화) → `norm` 라벨(`y`)은 1.1절 정의 그대로.
- `build_feature_bank(x_norm)` — **밀집(dense) 자동 feature 뱅크**, 수백~수천 개의 후보를 만든다:
 - `norm / log(norm)`에 대해 길이 `[4..64]` 슬라이딩 윈도우 평균(`MASS_add_window_stats`), 1차 미분(`d1`)/로그미분(`dlog`)의 윈도우 평균·표준편차, 2차 미분(`d2`)의 평균·표준편차·최소·최대.
 - `add_haar_edges`: 블록 크기 `[2,4,8,12,16,24]`의 "뒤 절반 평균 - 앞 절반 평균"(Haar-웨이블릿류 엣지) — `norm/log` 각각.
 - 11개 명명된 구간(`segs`: `early/earlymid/pretrough/transition/trough/troughcore/recover/late/tail` 등)의 레벨 평균과, 그 구간들 사이의 **모든 쌍별 contrast(차)와 ratio(비)** — `level`과 `log-level` 각각.
 - 5개 범위(`ranges`)에서 `min/max/argmin/argmax/range`를 뽑고, 그로부터 "tail rebound height", "early-to-trough drop", "recovery fraction" 같은 파생 지표를 조합.
 - 6개 구간에서 1차 미분의 절대평균/표준편차/양의 비율/부호전환 횟수/근-평탄(near-flat) 개수·런(run)·최장 런 등 "곡선이 얼마나 거친가/평평한가"를 재는 지표.
 - 전역 지표(표준편차, 범위, 왜도/첨도, 절대 기울기·곡률 평균), DCT 계수 48개, FFT 진폭 48개, 자기상관(다양한 lag 17개).
- `build_visual_norm_bank(norm)` — **시각적 해석에서 착안한 템플릿 feature**: `early/mid/tail` 3구간의 평균을 조합해 "trough depth"(`0.5*(early+tail) - mid`), "mid flatness"(`|mid - 0.5*(early+tail)|`), "tail - mid"를 다양한 구간 경계 조합에 대해 계산 + 1.2차 미분 기반 거칠기 지표.
- `build_candidate_bank()`: 위 두 은행을 `smooth_norm` / `smooth_visual` 접두어로 합침 — **"raw-level feature는 이 lock 프로토콜에서 아예 쓰지 않는다"**(주석 400행, 정규화된 형태만 사용).

이 feature 은행은 12절에서 설명하듯 **internal(train)만으로 통계(중앙값/분위수/평균/표준편차)를 낸 뒤 external에 그대로 적용**되며(`standardize_train_test`), 1-99% 분위수로 clip 후 z-표준화하고 표준편차가 거의 0인 컬럼은 제거한다.

4. 임상 모델 (line 99-171, 422-431)

- `clinical_matrix()`: `PatientAge`, `Height`, `Weight`, `sex_M` 4변수, internal 중앙값/평균/표준편차로 표준화 후 external에 고정 적용 — 다른 파이프라인과 동일 패턴.
- `clinical_estimator()`: `LogisticRegression(C=1e6)` — 사실상 무정규화.
- `oof_and_external()`: 5-fold OOF(internal) + fold별 모델로 만든 external 점수 평균(주의: 아래 `crossfit_candidate`의 "final+fold 평균 블렌드"와 달리, 여기서는 **최종 재학습 모델의 external 점수**를 그대로 반환 — fold 앙상블이 아님).
- `zfit_apply()` / `threshold_for_min_sensitivity()`: 임상 z-표준화 및 5개 OP(S80~S90)별 임계값 산출, `full_derivation` 파이프라인과 동일 로직.

5. Stage 1 — LOCK_main(): internal 전용 feature 잠금 (line 397-897)

5.1 사전 스크리닝 (prescreen_feature_indices, 453-472행)

- 임상 z-점수만으로 로지스틱 회귀를 적합해 $\text{residual}(y - \text{predict})$ 을 구하고, 전역 상관 $|x^T \text{residual}| / ||x||$ 과, 5개 OP별 clinical-positive 부분집합 내부에서의 상관을 합산한 cp_score 를 가중 합($\text{global} + 0.7 * \text{cp_score}$)한다.
- feature 이름에 curv/slope/haar/trough/waviness/dct/autocorr 가 포함되면 +0.08 보너스(semantic prior) — 형태 기반 feature를 약간 우대.
- 상위 MAX_FEATURES_SCREEN=600 개만 남겨 이후 단계의 계산량을 줄인다.

5.2 위험 방향(risk_direction, 442-451행)

- 임상 z만으로 로지스틱을 적합한 residual과 각 feature의 상관 부호(sign)를 구해 "이 feature가 커질 때/작아질 때 어느 쪽이 저-SMI 방향인지"를 통일한다($x_risk = x * \text{direction}$, 이후 모든 게이트는 "값이 크면 위험(=AEC+ 완화 방향)"으로 해석 가능해짐). 상관이 정확히 0이면 단순 event-rate 차이로 풀백.

5.3 단일 feature 스크리닝 (feature_screen, 581-601행)

- 600개(사전 스크리닝 후) feature $\times$ WIDTHS (3) $\times$ LAMBDA (4) 조합 각각에 대해 5개 OP에서 make_single_deesc(=full_derivation의 branch_gate_score + 임계값 비교와 동일한 게이트 공식)를 적용한 완화 결과를 계산.
- 생존 조건(fail): $\min_p_loss < 0.05$ 또는 $\min_spec_gain \leq 0$ 또는 $\max_fisher_p \geq 0.05$ 또는 $\min_deesc_n < 25$ 또는 $\max_sens_loss > 0.08$ 중 하나라도 해당하면 탈락 취급(score -= 10.0 으로 크게 감점, 완전히 제거하지는 않고 순위만 맨 뒤로 보냄).
- 점수식: $2.5 * \min_spec_gain + 1.0 * \text{mean_spec_gain} + 0.8 * \min_ba_delta - 0.45 * \max_sens_loss - 0.05 * \text{company_eta2}$ — company_eta2 (제조사별 분산 설명력)가 높을수록 감점 = 스캐너 제조사에 좌우되는 feature를 회피하려는 설계.

5.4 다양성 있는 후보 풀(diverse_combo_pool, 620-662행)

- 점수 내림차순으로 훑으며 feature family당 최대 5개(feature_family: shape_contrast/curvature/absolute_slope/signed_slope/haar/spectral/level/other)만 허용하고, 이미 뽑힌 feature와 상관계수 0.92 이상이면 스킵(중복 정보 배제) — TOP_FEATURES_FOR_COMBO=18 개를 채울 때까지.
- 18개를 못 채우면(다양성 조건이 너무 빡빡하면) 조건을 풀고 점수 순으로 채움(651-661행).

5.5 조합 탐색(combo_search, 689-718행) — 확정 규칙이 나오는 곳

- 18개 후보 중 $m=1..MAX_COMBO_M(4)$ 개를 뽑는 모든 조합($\text{itertools.combinations}$)에 대해:
 - $m=1$ 이면 $k=1$ 만, 그 외엔 $k \in [(m+1)//2 .. m]$ (과반수~전원 일치) 범위의 "k-of-m" 규칙을 평가.
 - internal에서만(["gangnam_internal"]) 지표를 계산 → LOCK_summarize_internal().
 - 공식 비열등성 검정: 해당 조합의 S90 행에서 tp_lost (놓친 저-SMI 환자 수)로 Clopper-Pearson 단측 95% 상한을 구해 FORMAL_NI_MARGIN(0.05) 이하인지(formal_ni_pass) 확인 — 이게 5.3절의 완만한 $\min_p_loss \geq 0.05$ 기준과 별개로 S90 하나에 대해서만 추가로 강제되는, clinical_vs_aec_assisted_table.png 와 동일한 엄격한 판정이다.
 - survives = (5.3절과 동일한 5개 조건) AND formal_ni_pass.
 - 점수식은 5.3절과 유사하되 m (feature 개수)이 클수록 미세 가산($+0.01 * \min(m, 3)$), 복잡도가 크게 선호되지 않도록 상한을 둬)하고 company_eta2 평균에 대한 페널티 계수는 더 작음(-0.04 vs -0.05).
 - survives_internal_constraints → lock_selection_score 내림차순 정렬 후, 조건을 만족하는 최상위 1개(없으면 그냥 최상위 1개)를 **잠긴 규칙(locked_row)**으로 채택.

5.6 잠긴 규칙 적용 및 부가 분석

- locked_details: 잠긴(subset, k)를 internal 및 external 양쪽 5개 OP에 적용해 재계산(외부는 오직 평가 목적).

- `adjusted_p_for_row()` : 제조사(스캐너) 더미변수 + (옵션) 임상 z를 넣은 로지스틱 회귀에서 "완화 여부" 계수의 우도비검정(LRT) p값 — **완화 효과가 스캐너 제조사나 임상 점수로 설명되는 교란이 아님**을 보이기 위한 보정 분석(`scanner_only` vs `scanner_plus_clinical` 두 버전).
- `locked_score_auc_table()` : "잠긴 feature들의 단순 평균"을 AEC 점수로 삼아 임상 점수와 결합한 로지스틱(`C=1.0`, `LOCK_SEED` 기반 5-fold)의 AUC를, `clinical_only` / `locked_aec_score_only` / `clinical_plus_locked_aec_score` 세 모델로 비교.
- 산출물: `locked_gate_features.csv` (잠긴 feature 이름/width/λ), `locked_gate_summary.json` (잠긴 규칙 전체), `locked_gate_operating_point_details.csv`, `locked_gate_adjusted_pvalues.csv`, `locked_gate_auc_summary.csv`, `locked_gate_operating_points.png` (2x2: 코호트별 특이도 곡선 + 특이도 개선/민감도 손실 막대).

6. Stage 2 — `PATTERN_main()` : region-guided CNN이 잠긴 규칙을 근사 (line 899-1394)

6.1 왜 CNN인가

Stage 1의 잠긴 규칙은 "특정 4개 feature의 손수 계산한 값"에 의존한다. Stage 2는 그 규칙을 **정답 레이블**로 삼아, 곡선 원본(넓은 원도우)에서 직접 그 판정을 근사하는 작은 CNN을 학습한다 — 즉 "손수 만든 4개 feature가 정말 곡선의 그 근방 모양만 보고도 재현 가능한 신호인가"를 검증하는 동시에, 탐색으로 고정된 정밀 경계 대신 학습된 표현으로 일반화 여지를 넓히는 단계다.

- `REGIONS` (2절)는 Stage 1이 고른 feature의 이름에 들어있는 좁은 구간(예: `082_085`) **주변을 넓힌 원도우**다 — CNN이 정확한 경계를 강제로 배우지 않고 근방의 형태로부터 특징을 스스로 추출하게 함.
- `make_channels()` : `norm`, `d1(norm)`, `d2(norm)` 을 각각 **환자별로 z-정규화**(`row_z`)한 3채널 입력 — "곡선의 절대 레벨"이 아니라 "형태(morphology)"만 보게 강제.
- `standardize_channels_train_apply()` : 채널별 평균/표준편차를 internal로만 산출해 external에 적용.

6.2 아키텍처(`RegionBranch`, `DirectVoteCnn`, 928-1012행)

- `RegionBranch` : `Conv1d(3→8, k=5) → BN → SiLU → Conv1d(8→8, k=3) → BN → SiLU` 후 시간축 평균+최댓값을 `concat(hidden*2)`해 `Linear(→1)` 로 스칼라 점수화 — 4개 region마다 독립된 branch.
- `DirectVoteCnn.forward()` : 4개 branch 점수(`morph`)를 얻은 뒤, **full_derivation**의 **branch_gate_score**와 동일한 구조의 **feature 집합**(`morph`, `morph*boundary`, `delta`(=`clinical_z-th`), `boundary`, `cpos`)을 만들어 `head_weight` (4 region × 5 feature) 선형결합 + `head_bias` 로 5개 OP × 4 region 로짓을 산출.
- 초기화가 분석적 규칙에서 시작(994-999행): `head_weight[:,1]=-1.5` (`morph*boundary` 항), `[:,2]=-2.0` (`delta` 항), `[:,4]=0.5` (`cpos` 항), `bias=-1.0` — 무작위 초기화 대신 "임상 점수가 낮을수록/경계 근처 boundary가 클수록/CNN 형태 점수가 낮을수록 +로 투표"라는 분석적 게이트에 가까운 지점에서 학습을 시작해 수렴을 돕는다.

6.3 학습 타깃과 손실

- `exact_feature_votes()` : Stage 1이 잠근 feature들(`locked_targets()` 가 `locked_gate_features.csv` 에서 이름을 읽어와 feature 은행을 재계산 후 해당 컬럼만 추출)에 `BRANCH_WIDTH` / `BRANCH_LAMBDA` (2절, Stage 1의 탐색된 width/λ가 아니라 CNN 타깃 생성 전용 고정값)로 `make_single_deesc()` 를 적용한 ****분석적 투표 결과(0/1)****를 "정답"으로 삼는다.
- `loss_fn()` : (a) `branch_loss` — OP별 클래스 불균형에 맞춘 `pos_weight` (양성 라벨이 희소하므로 최대 30배 가중)로 4개 branch 각각의 BCE, clinical-negative 샘플은 `non_cpos_weight` (0.03~0.05)로 대폭 낮은 가중치를 줌(애초에 clinical+ 안에서만 의미 있는 게이트이므로). (b) `consensus_loss` — `soft_atleast2_prob()` (4개 중 "정확히 0개" 확률과 "정확히 1개" 확률을 배제한 여집합, 즉 확률적 "2개 이상 +" 근사)과 실제 "2개 이상 +"였는지의 BCE. 최종 손실은 `branch_loss + consensus_weight * consensus_loss`.
- `DVOTE_train_one_fold()` : AdamW, 최대 180epoch, 검증손실 기준 조기종료(patience=20), 매 시드-폴드마다 전부 재시딩(`np.random.seed`, `torch.manual_seed`, 결정적 알고리즘 강제).

- `DVOTE_crossfit_config()`: `DVOTE_SEEDS` (2개) × `StratifiedKFold(5)` 로 OOF/external 로짓을 얻고, 두 시드의 결과를 평균 — 단일 학습 런의 변동성을 줄임.

6.4 확률 → 패턴 게이트 재탐색(`search_pattern_gate` , `load_or_train_probabilities`)

- `PROB_CACHE` (npz)가 있으면 재학습을 건너뛰고 캐시된 확률을 읽음 — CNN 학습은 느리므로 반복 실행 시 캐싱.
- `threshold_vectors()`: 4-region 공통 임계값(0.35~0.85, 0.05 간격) + `{0.55,0.65,0.75}^4` (4개 region이 서로 다른 임계값을 가질 수 있는 조합) 격자.
- 각 임계값 벡터에서 확률을 4-bit 코드(0~15)로 변환(`codes_from_prob`) 후:
 1. `rank_single_patterns()`: 16개 단일 코드 각각을 "+로만 인정"했을 때의 빠른 근사 점수 (`fast_summary_internal` , Fisher 검정 없이 이항 근사)로 순위를 매겨 상위 6개(`top_codes`) 선정.
 2. `candidate_masks()`: 16개 단일코드, `popcount≥k` 합집합($k=1..4$), `popcount==k` 합집합($k=1..4$), 그리고 상위 6개 코드 중 2~3개 조합 — 다양한 크기/모양의 패턴 부분집합 후보 생성.
 3. 모든 (임계값 벡터, 후보 마스크) 조합을 `fast_summary_internal` 로 1차 스코어링(`internal_score` : `min_p_loss>=0.05`, `max_sens_loss<=0.08`, `min_spec_gain>0`, `fisher(있으면)<0.05`, `min_deesc_n>=25`, `mean_event_rate<=0.12` 생존 조건 + 가중합 점수) 후 상위 300개만 정확한 `evaluate_pattern_gate / PATTERN_summarize_internal` (Fisher 정확검정 포함)로 재평가 — 전수 정확 평가는 비용이 크므로 2단계(근사 → 정확) 필터링.
 4. 두 `DVOTE_CONFIGS` (balanced/guarded) 각각에 대해 최선의 (임계값, 패턴) 조합을 뽑고, 그중 더 좋은 설정을 최종 채택.
- 비교 기준선 `exact_locked_2of4`: Stage 1 잠긴 feature의 분석적 투표(CNN 확률이 아니라 정답 라벨 자체)에 "4개 중 2개 이상 +"를 적용한 규칙 — CNN 기반 패턴 게이트가 이 분석적 기준선 대비 얼마나 같거나/다른 성능을 내는지 그림으로 나란히 비교(`plot_best`).
- 산출물: `pattern_gate_summary.json` (선택된 config/임계값/패턴), `pattern_gate_best_deescalation_details.csv` , `pattern_gate_best_pattern_distribution.csv` , `pattern_gate_best_plot.png` , `direct_vote_probabilities.npz` (Stage 3 입력이 되는 확률 캐시).

7. Stage 3 — `BOOST_main()`: CNN 확률 위에 ML을 얹어 AUC 개선 탐색 (line 1396-1629)

이 단계는 "게이트 규칙"이 아니라 **AUC를 높일 수 있는지의 탐색적 분석**이다(주석: "Exploratory AUC-max calibration. External AUC is the only relevant target for portability").

7.1 입력 feature 구성(`direct_vote_features` , `build_base_features` , 1459-1505행)

Stage 2의 확률 텐서($N \times 5 \text{ OP} \times 4 \text{ region}$)로부터:

- 원본 확률을 그대로 펼친 것(`flat`),
- `soft_atleast2_prob` (OP별 "2개 이상 +" 확률)과 그 평균/최소/최대/표준편차(OP 축 통합),
- region 축 평균/표준편차, OP 축 평균/표준편차,
- 6개 임계값(0.50~0.75)마다: 이진 투표, OP별 투표 개수, "개수≥2" consensus 플래그.

`feature_set_matrix()` 가 4가지 조합을 선택:

- `vote`: 위 base feature 그대로.
- `clinical_vote`: base + `add_clinical_features()` (임상 점수/ $z/z^2/z^3$ /OP별 delta-boundary-cpos flag).
- `vote_poly`: base 앞 80열의 2차 상호작용(교차항만, `interaction_only=True`).
- `clinical_vote_poly`: `soft2_ / branch / clinical_` 관련 열만(최대 120개)로 상호작용 — 상호작용 폭발을 억제하기 위한 사전 필터.

7.2 후보 모델 13종과 crossfit

- CANDIDATES (2절)는 `vote / vote_poly / clinical_vote / clinical_vote_poly` feature 세트와 `logit_l2/logit_l1(+SelectKBest40)/svm_rbf(+SelectKBest60)/histgb/rf/extratrees` 모델의 조합.
- `crossfit_candidate()`: `StratifiedKFold(5, BOOST_SEED)` 로 OOF 계산, **external은 "5-fold 각각의 external 예측 평균"과 "internal 전체로 재학습한 최종 모델의 external 예측"을 50:50 블렌드** (`ext = 0.5*ext_final + 0.5*mean(ext_scores)`) — Stage 1 `oof_and_external` (재학습 단일 모델만 사용)과 달리 분산을 줄이기 위해 앙상블을 섞음.
- `paired_delta_bootstrap()`: 후보 점수 - 임상 점수의 AUC 차이를 2000회 부트스트랩으로 재표본해 양측 p값과 95% CI를 구함(관측값이 정확히 0이면 부호가 나뉘어 $p=1$ 에 가까워지도록 `2*min(mean<=0, mean>=0)`).
- `raw_low_smi_risk` (1577-1588행): 학습 없이 `soft_atleast2_prob` 평균만으로도 순위를 매길 수 있는 가장 단순한 베이스 라인도 함께 비교.
- 산출물: `direct_vote_auc_boost_summary.csv` (모델별 internal/external AUC, 임상 대비 델타 + 부트스트랩 p/CI), `direct_vote_auc_boost_scores.csv` (**환자별** 원점수 — Stage 4가 여기서 `vote_only_logit_l1` 컬럼을 읽음), `direct_vote_auc_boost_plot.png` (가로 막대: 모델별 internal/external AUC, 임상 기준선과 AUC 0.90 목표선 표시).

8. Stage 4 — `FINAL_main()`: 임상 고위험군 내 AEC 표현형 enrichment (line 1631-1941)

8.1 이 단계의 주장이 다른 단계와 다른 점

앞의 세 단계는 전부 "완화(de-escalation) 게이트"나 "AUC 개선"을 다뤘다. Stage 4는 명시적으로 **진단 성능 주장이 아니다**(주석 1631-1638행, `final_summary.json` 의 `important_caution`): "AEC는 임상 저-SMI 모델을 대체하지 않는다. 이미 임상적으로 고위험인 환자들 중에서, AEC 형태 점수가 저-SMI가 농축된(enriched) 하위군과 그렇지 않은 하위군을 분리한다"는 **표현형 계층화(phenotype stratification)** 주장이다.

8.2 입력 준비

- `run_if_needed()`: `direct_vote_auc_boost_scores.csv` 가 없으면 과거의 독립 스크립트 3개(`work/aec_*.py`) 를 순서대로 서브프로세스 실행해 재생성하려 시도한다 — **다만 서두에 적었듯 이 저장소에는 `work/` 디렉터리 자체가 없으므로, 이 fallback 경로는 현재 사용 불가능하고 `main()` 으로 Stage 1~3을 먼저 실행해 CSV를 만들어 줘야 한다.**
- `load_patient_table()`: 두 코호트 `metadata` (`load_metadata`, TAMA/IMATA/BMI 등 파생 지표 포함)를 Stage 3의 점수 CSV 와 (`cohort`, `row_index`) 로 병합, 예상 표본 수(internal 1090 / external 926)와 다르면 경고만 출력(하드 실패는 아님).

8.3 분위수 플레그(`add_global_flags`, 1765-1786행) — 핵심 설계

모든 컷오프는 **internal 코호트에서만 계산한다**:

- `clinical_cut`: internal 임상 점수의 상위 q ($=0.20$) 분위수 → `clinical_pos` (전체 데이터에 적용).
- `aec_global_cut`: internal AEC 점수의 상위 q 분위수 → `aec_pos_global`.
- 주 분석 대상**: `aec_high_in_clinical_pos / aec_low_in_clinical_pos` — **internal의 clinical-positive 부분집합 내부에서** AEC 점수 상/하위 q (즉 "임상 고위험군 중에서도 AEC 형태가 위/아래로 극단인 사람들")를 정의하고, 이 컷오프를 전체(외부 포함) 데이터에 그대로 적용.
- 네 칸 분류(`cell`): `C+A+ / C+A- / C-A+ / C-A-` (전역 AEC+ 기준).

8.4 표

- `enrichment_table()` — **주 결과표**: clinical-positive 내부에서 AEC-high vs AEC-low의 저-SMI 발생률, 절대위험차/위험비/오즈비, Fisher 정확검정(**단측**, `alternative="greater"` — "AEC-high가 AEC-low보다 높다"는 방향성 가설을 사전에 고정).
- `or_and_table()` — 보조표: `Clinical+ / AEC+ global / C OR A / C AND A` 네 규칙의 통상적 진단 지표 (민감도/특이도/PPV 등) — 저자 주석: "secondary table (not the primary message)".

- `cell_characteristics()` — 네 칸(C+A+ 등)별 인구통계·점수 평균/표준편차.
- `low_smi_subtype_tables()` — **실제 저-SMI 환자만** 걸러 AEC+ vs AEC-(및 C+A+ vs 나머지)의 인구통계 차이를 Mann-Whitney U(연속형)/Fisher(성별)로 검정 — "AEC가 저-SMI 안에서 어떤 하위표현형을 잡아내는지" 탐색(주석: 과거 관찰 — AEC+ 저-SMI 환자가 더 마름/낮은 BMI·체중/높은 TAMA-per-weight 경향).

8.5 그림 & 산출물

- `plot_quintile_enrichment()`: 코호트별로 "Clinical high / AEC low / AEC high" 3개 막대의 관측 저-SMI 유병률(% , n 라벨 포함) — 8.1절 주장을 한 장으로 요약.
- CSV 6종(`00_patient_level_merged_scores.csv` ~ `05_low_smi_subtype_feature_tests.csv`) + `final_summary.json` (파라미터/주장/주의사항/산출 경로 전부 기록).

9. MD 재현성 점검 카드 (`CHECK_main`, line 1943-2066)

- `CHECK_REFERENCE`: 협업자로부터 전달받은 원본 수치($q=20\%$ 기준, internal/external 각각의 AEC-low/-high 분모·분자·Fisher p) — 코드로 재계산 불가능한 "예전에 보고된 값"을 상수로 고정.
- `CHECK_verdict()`: 분자/분모가 정확히 같으면 "일치", 비율 차이 $2\%p$ 이내면 "근사일치", 그 외 "불일치".
- `CHECK_main()` 은 `01_quintile_vs_quartile_enrichment.csv` 의 $q=0.20$ 행을 참조값과 나란히 비교해 `outputs/run_from_raw_standalone/MD/quintile_enrichment_reproduction_check.png` 카드를 만든다.
- **코드 자체에 이미 알려진 불일치가 문서화되어 있다**(2062행 footer): external의 AEC-low(12/54)는 정확히 일치하지만, AEC-high 표본이 재현 결과(78명)가 참조값(69명)보다 커서 p-value가 더 작게 나옴 — 저자는 이를 "두 표본에 공통 적용되는 internal-locked AEC-high 컷오프 자체가 참조 파이프라인과 달랐을 가능성"으로 원인을 남겨둠. MDCARD (다른 파이프라인)와 달리 이 카드는 **완전한 일치를 주장하지 않고, 알려진 차이의 원인 후보까지 그대로 카드에 기록**한다는 점이 특징이다.

10. 재현성을 위한 전역 RNG 재시딩 (`reset_shared_random_state`, `stage_action`, line 2067-2078)

`main()` 이 4단계를 실행하기 직전마다 `stage_action()` 래퍼가 전역 RNG 를 `np.random.default_rng(COND_SEED)` 로 새로 만든다. 이는 각 단계(`LOCK_main`, `PATTERN_main` 등)가 내부에서 `make_folds()` (전역 RNG 소비)를 호출하는데, **이전 단계가 이미 RNG 를 얼마나 소비했는지와 무관하게 항상 동일한 fold 분할이 나오도록** 보장하기 위함이다 — `full_derivation_pipeline_simplified.py` 가 `make_context()` 호출마다 재시딩하는 것과 동일한 문제의식.

11. 데이터 누수 방지 설계 요약

- 임상/feature 표준화 파라미터(중앙값·평균·표준편차·1-99% clip 경계), 사전 스크리닝 상위 feature, 단일/조합 feature 선택, k-of-m 규칙, CNN 학습 타깃과 하이퍼파라미터 선택, 패턴 게이트 임계값, Stage 4의 모든 분위수 컷오프 — **전부 internal(gangnam) 코호트에서만 결정**되고 external(sinchon)에는 고정 적용만 된다.
- Stage 1 `combo_search` 와 Stage 2 `search_pattern_gate` 의 생존 조건(`survives_internal_constraints`)은 **internal 지표만으로 판정**한다(`full_derivation` 파이프라인이 internal+external 양쪽 통과를 요구하는 것과 달리, 이 standalone 스크립트는 external을 사후 확인/보고 용도로만 쓰고 선택 기준에는 포함하지 않음 — 두 파이프라인의 검증 엄격도 차이로 유의할 부분).
- Stage 3의 부트스트랩-crossfit은 internal 5-fold CV로만 모델을 학습하고, external은 각 fold 모델의 예측을 모아 평가에만 사용한다.
- Stage 4의 `load_patient_table()` 은 병합 후 표본 수를 하드코딩된 기대값과 대조해 데이터 정합성을 경고 수준으로 점검한다.